Expt. No. 10

***Title**: Exploratory Data Analysis (EDA)*

***Name:** Girish Sakpal*

***UIN:** 231A011*

Rubric	***Score (0 to 5)***
In time Submission	
Clean, Commented and Correctness of code (Screenshot with Name and UIN)	
Concept Understanding (ability to answer viva)	
Total	

***Performed On:** 08/10/2026*

***Sign:** _____*

Experiment No. 10

Aim: Perform exploratory data analysis on a text corpus, visualizing important insights with tools like word clouds, frequency plots, and distribution plots

Pre-Lab Question:

1. **What is exploratory data analysis (EDA) in the context of a text corpus?**
   EDA in a text corpus is the process of examining and understanding the text data using statistics and visualizations. It helps identify patterns, structures, and characteristics of the text.
2. **What insights can be gained by performing EDA on text data?**
   EDA can reveal frequently used words, document length, word distributions, common topics, and relationships between terms. It also helps identify patterns and unusual data.
3. **Discuss some of the important insights that can be obtained, such as common terms, word frequency distributions, or patterns of occurrence.**
   Common terms show the main topics in the corpus, while word-frequency distributions show how often words occur. Patterns of occurrence can reveal important relationships and recurring themes.
4. **Discuss the process of tokenization and its importance in analyzing a text corpus.**
   Tokenization breaks text into smaller units such as words or sentences called tokens. It is important because these tokens can then be analyzed for frequency, patterns, and other linguistic features.
5. **What are stop words, and why is it important to remove them before visualization?**
   Stop words are common words such as *the, is, and, of,* and *a* that usually carry little meaning. Removing them makes visualizations clearer and highlights more meaningful words.
6. **What tools or Python libraries are typically used for visualizing text data?**
   Common Python libraries include Matplotlib, Seaborn, WordCloud, and Plotly. They can be used to create word clouds, frequency charts, bar graphs, and other visualizations.

Post-Lab Question:

1. **What were the most frequently occurring words in the text corpus?**
   The most frequently occurring words were the meaningful terms related to the main topic of the corpus. Common words appeared more often than less relevant or rare terms.
2. **How did stop word removal affect the visualization results, such as the word cloud?**
   Removing stop words made the word cloud clearer by eliminating common words like *the, is, and,* and *of*. This highlighted the important and meaningful terms.
3. **Explain the impact of removing stop words on the word cloud or frequency plot.**
   It reduced noise and improved the readability of the visualizations. The frequency plot and word cloud focused more on topic-specific words.
4. **What patterns did you observe in the frequency plots or distribution plots?**
   A small number of words occurred very frequently, while many other words appeared less often. This showed a typical uneven or long-tail distribution of word frequencies.
5. **Compare the utility of word clouds with other visualization techniques like frequency or distribution plots.**
   Word clouds provide a quick visual overview of common words, while frequency and distribution plots give more accurate quantitative information about word occurrences and comparisons.
6. **How would EDA help in subsequent NLP tasks such as sentiment analysis or classification?**
   EDA helps identify important words, patterns, and noise in the data before modeling. This can improve preprocessing, feature selection, and the performance of sentiment analysis or classification models.

Software Used:

Theory:

Exploratory Data Analysis (EDA) in NLP:

1. EDA is a technique used to summarize the main characteristics of a dataset using both statistical and graphical methods. For text data, EDA involves analyzing the content to discover patterns, identify relationships, and understand the underlying structure of the text.
2. The goal of EDA in a text corpus is to transform unstructured data into meaningful information that can help guide further processing or model building.

Text Preprocessing:

1. Tokenization: Tokenization is the process of breaking down the text into smaller units, such as words or phrases. This is a critical preprocessing step that allows further analysis on individual components of the text.
2. Stop Word Removal: Stop words are common words like "the," "is," and "and" that do not carry significant meaning. Removing them reduces noise and allows important content words to stand out during visualization.
3. Normalization: Text normalization techniques like lowercasing, stemming, and lemmatization are used to reduce word variants to a base form. This makes word analysis more effective by grouping similar words.

Visualization Tools:

1. Word Clouds: A word cloud is a visual representation of the most frequently occurring words in the text corpus. Larger words in the word cloud indicate higher frequencies.

   Word clouds are an effective way to quickly understand the dominant terms and themes within a text dataset.
2. Frequency Plots: Frequency plots display how often different words occur in the corpus. These plots can reveal the distribution of word usage, highlighting common terms and helping detect skewed frequency distributions.

   Frequency analysis is essential in determining which words are key to understanding the content and which are just filler.
3. Distribution Plots: Distribution plots help visualize the overall distribution of word lengths, sentence lengths, or other numerical properties of the text data.

   These plots can reveal patterns that indicate how the language is used, such as the complexity of the vocabulary or consistency in sentence structure.

Analyzing Text Features:

1. Term Frequency: Counting word occurrences provides an understanding of the most important words in the corpus. High-frequency terms can indicate major topics or themes.
2. N-grams Analysis: N-grams (unigrams, bigrams, trigrams, etc.) can be extracted from the text to analyze common word combinations and phrases, which can help uncover recurring patterns and collocations.

Importance of EDA in NLP:

1. Understanding Data: EDA provides insights into the data that are critical for choosing appropriate preprocessing techniques, designing features, and selecting NLP models.
2. Identifying Bias and Imbalance: By visualizing the distribution of words or terms, EDA can help identify biases, imbalance in the dataset, or topics that are underrepresented.
3. Feature Engineering: Insights from EDA can guide feature engineering, such as selecting the most meaningful words for further tasks like classification, sentiment analysis, or clustering.

Tools for Visualization:

1. WordCloud Library: Used to create word clouds that visually represent word frequencies.

2. Matplotlib and Seaborn: Libraries like Matplotlib and Seaborn are used for plotting frequency and distribution plots, making it easier to analyze trends in the text data.

3. Pandas and NLTK: Pandas for data manipulation and NLTK (Natural Language Toolkit) for text preprocessing are often used in conjunction with visualization libraries to facilitate EDA.

Algorithm:

Input: Text corpus (collection of documents).

Step 1: Text Preprocessing:

a. Tokenization: Split text into individual words or tokens.

b. Lowercasing: Convert all text to lowercase for uniformity.

c. Stop Word Removal: Remove common stop words (e.g., "is," "and," "the") to focus on content rich words.

d. Normalization: Perform stemming or lemmatization to reduce words to their root forms (optional).

Step 2: Frequency Analysis:

a. Calculate Term Frequency: Count the occurrences of each word in the corpus. b. Extract N-grams:

Extract unigrams, bigrams, or trigrams for analyzing common phrases. Step 3: Data Visualization:

a. Word Cloud: Generate a word cloud to visualize the most frequent words in the text corpus. b.

Frequency Plot: Plot word frequencies using a bar plot to visualize the most common words.

c. Distribution Plot: Plot the distribution of word lengths or sentence lengths using a histogram or distribution plot.

Step 4: Visualization Analysis:

a. Interpret Word Cloud: Analyze which words appear most prominently and why.

b. Frequency and Distribution Insights: Observe which words dominate the text and how word or sentence lengths are distributed.

Output:

Visualizations (word cloud, frequency plots, distribution plots) and insights about the text structure and key themes.

Example

Input Corpus: "The cat sat on the mat. The dog sat too."

Output: Word Cloud highlighting "cat," "dog," and "sat" prominently.

Frequency plot showing word frequencies, e.g., "sat" occurring most often.

Distribution plot for word lengths indicating most words have 3-4 letters.

Program and Output:

```
print("Performed By Girish Sakpal(UIN: 231A011)")
import pandas as pd
from datasets import load_dataset
from sklearn.feature_extraction.text import CountVectorizer
import matplotlib.pyplot as plt
from collections import Counter
from nltk import ngrams
import string
from wordcloud import WordCloud
import re

# Load complete IMDB dataset
dataset = load_dataset("stanfordnlp/imdb")
```

```
Performed By Girish Sakpal(UIN: 231A011)
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests t
README.md: 100% 7.81k/7.81k [00:00<00:00, 156kB/s]
plain_text/train-00000-of-00001.parquet: reconstructing file: 100% 21.0MB / 21.0MB, 2.05MB/s
plain_text/train-00000-of-00001.parquet: downloading bytes: 20.9MB, 2.04MB/s
plain_text/test-00000-of-00001.parquet: reconstructing file: 100% 20.5MB / 20.5MB, 2.01MB/s
plain_text/test-00000-of-00001.parquet: downloading bytes: 20.4MB, 2.00MB/s
plain_text/unsupervised-00000-of-00001.p(...): reconstructing file: 100% 42.0MB / 42.0MB, 4.09MB/s
plain_text/unsupervised-00000-of-00001.p(...): downloading bytes: 41.9MB, 4.06MB/s
Generating train split: 100% 25000/25000 [00:00<00:00, 57349.69 examples/s]
Generating test split: 100% 25000/25000 [00:00<00:00, 46324.03 examples/s]
Generating unsupervised split: 100% 50000/50000 [00:00<00:00, 69738.64 examples/s]
```

```
print("Performed By Girish Sakpal(UIN: 231A011)")
#Combine and train an test
df = pd.concat(
    [dataset["train"].to_pandas(),
     dataset["test"].to_pandas()],
    ignore_index=True
)
#Rename Target Column
df = df.rename(columns={"label": "labels"})
```

```
print(df.shape)
print(df.columns)
print(df["labels"].value_counts())
```

```
Performed By Girish Sakpal(UIN: 231A011)
(50000, 2)
Index(['text', 'labels'], dtype='object')
labels
0    25000
1    25000
Name: count, dtype: int64
```

```
print("Performed By Girish Sakpal(UIN: 231A011)")
df.head()
```

```
Performed By Girish Sakpal(UIN: 231A011)
```

	text	labels
0	I rented I AM CURIOUS-YELLOW from my video sto...	0
1	"I Am Curious: Yellow" is a risible and preten...	0
2	If only to avoid making this type of film in t...	0
3	This film was probably inspired by Godard's Ma...	0
4	Oh, brother...after hearing about this ridicul...	0

```
print("Performed By Girish Sakpal(UIN: 231A011)")
#Check duplicate and missing values
print("Missing Values:")
print(df.isna().sum())
print("\nNumber of Duplicate Values:")
print(df.duplicated().sum())
```

```
Performed By Girish Sakpal(UIN: 231A011)
Missing Values:
text      0
labels    0
dtype: int64

Number of Duplicate Values:
418
```

```
print("Performed By Girish Sakpal(UIN: 231A011)")
#basic text statistics
df["char_count"] = df["text"].astype(str).apply(len)
df["word_count"] = df["text"].astype(str).apply(lambda x: len(x.split()))
df["unique_word_count"] = df["text"].astype(str).apply(
    lambda x: len(set(x.lower().split()))
)
df["avg_word_length"] = df["text"].astype(str).apply(
    lambda x: sum(len(word) for word in x.split()) /
    max(len(x.split()), 1)
)
```

```
print("Test Statistics:")
print(
    df[
        ["char_count",
         "word_count",
         "unique_word_count",
         "avg_word_length"]
    ].describe()
)
```

```
Performed By Girish Sakpal(UIN: 231A011)
Test Statistics:
         char_count    word_count  unique_word_count  avg_word_length
count  50000.000000  50000.000000       50000.000000     50000.000000
mean    1309.431020    231.156940         148.049780         4.640676
std      989.728014    171.343997          88.837364         0.340731
min       32.000000      4.000000           4.000000         1.239865
25%      699.000000    126.000000          91.000000         4.417904
50%      970.000000    173.000000         120.000000         4.627006
75%     1590.250000    280.000000         180.000000         4.847458
max    13704.000000   2470.000000        1092.000000        12.290909
```

```
print("Performed By Girish Sakpal(UIN: 231A011)")
def clean_text(text):
    #Convert to lowercase
    text = text.lower()
    #Remove HTML tags
    text = re.sub(r"<.*?>", "", text)
    #Keep alphabetic characters only
    text = re.sub(r"[^a-zA-Z\s]", "", text)
    #Remove extra whitespaces
    text = re.sub(r"\s+", " ", text).strip()
    return text

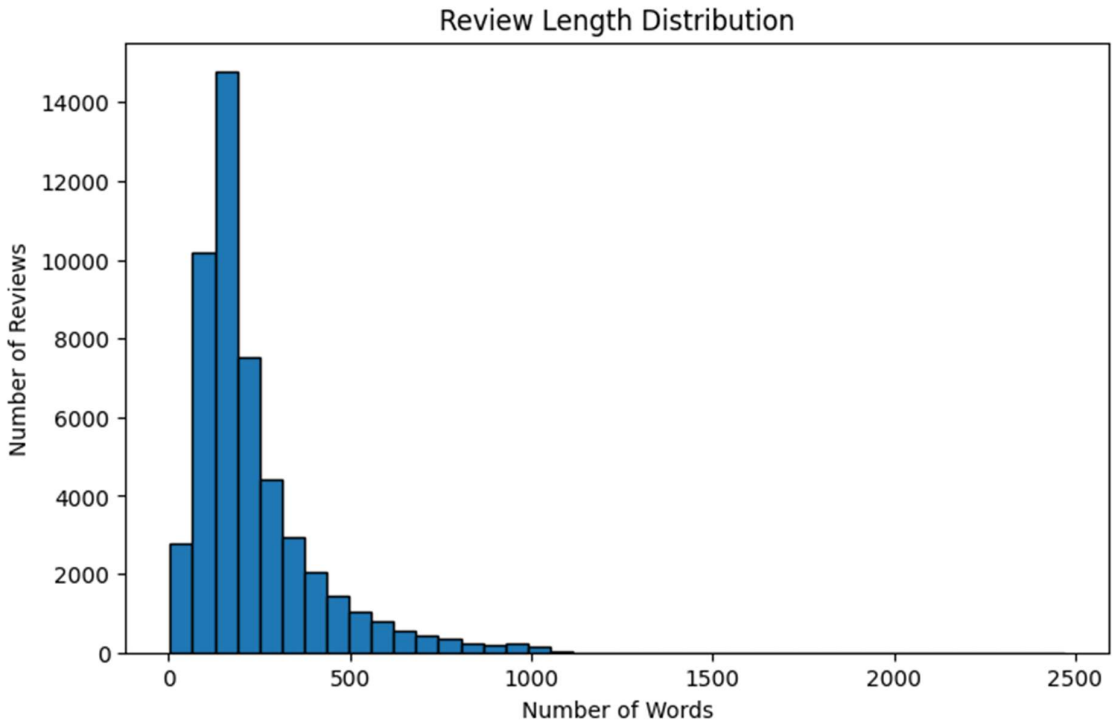
#Using 'text' instead of the non-existent 'review' column
df["clean_text"] = df["text"].astype(str).apply(clean_text)
print("\nOriginal Review:")
print(df["text"].iloc[10][:500])
print("\nCleaned Review:")
print(df["clean_text"].iloc[10][:500])
```

```
Performed By Girish Sakpal(UIN: 231A011)

Original Review:
It was great to see some of my favorite stars of 30 years ago including John Ritter, Be

Cleaned Review:
it was great to see some of my favorite stars of years ago including john ritter ben ga
```

```
print("Performed By Girish Sakpal(UIN: 231A011)")
#Document Length Distribution
plt.figure(figsize=(8,5))
plt.hist(df["word_count"], bins=40, edgecolor="black")
plt.xlabel("Number of Words")
plt.ylabel("Number of Reviews")
plt.title("Review Length Distribution")
plt.show()
```

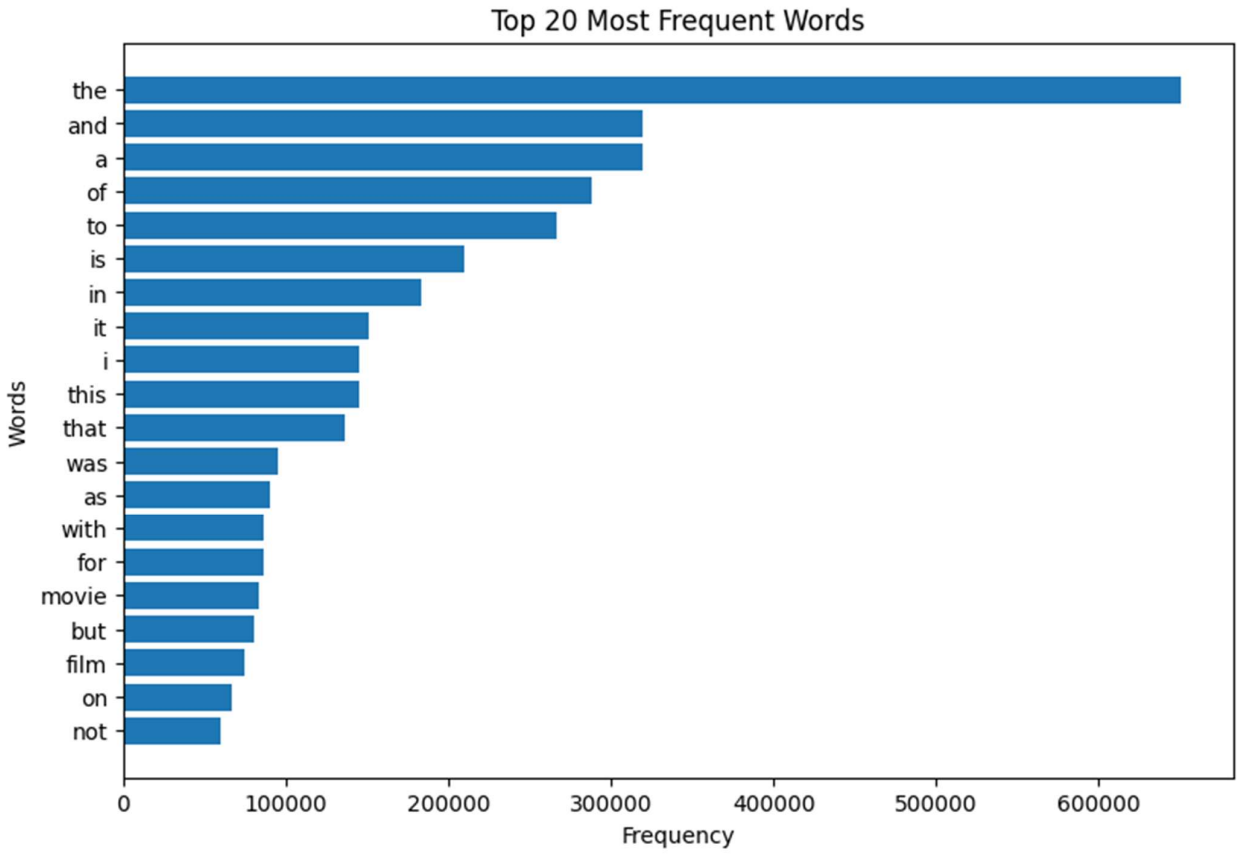



```
print("Performed By Girish Sakpal(UIN: 231A011)")
#Create word plot
all_words = " ".join(df["clean_text"]).split()
print("Total number of words:", len(all_words))
print("Vocabulary Size:", len(set(all_words)))
```

```
Performed By Girish Sakpal(UIN: 231A011)
Total number of words: 11259489
Vocabulary Size: 214620
```

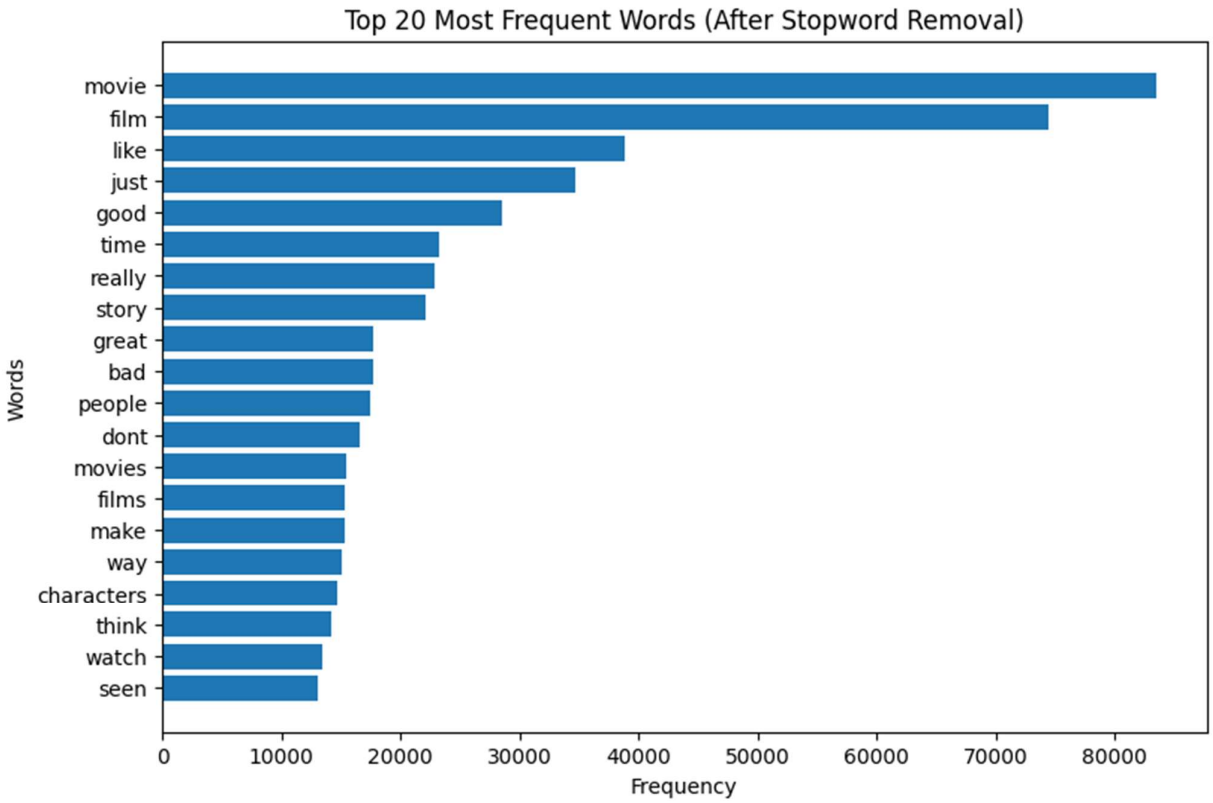
```
print("Performed By Girish Sakpal(UIN: 231A011)")
#Most Frequent Words
word_freq = Counter(all_words)
top_words = word_freq.most_common(20)
words = [word for word, count in top_words]
counts = [count for word, count in top_words]
plt.figure(figsize=(9,6))
plt.barh(words[::-1], counts[::-1])
plt.xlabel("Frequency")
plt.ylabel("Words")
plt.title("Top 20 Most Frequent Words")
plt.show()
```

```
Performed By Girish Sakpal(UIN: 231A011)
```




```
print("Performed By Girish Sakpal(UIN: 231A011)")
#Remove stopwords
vectorizer = CountVectorizer(
    stop_words="english",
)
vectorizer.fit(df["clean_text"])
stop_words = vectorizer.get_stop_words()
filtered_words = [word for word in all_words if word not in stop_words]
filtered_freq = Counter(filtered_words)
top_filtered = filtered_freq.most_common(20)
words = [word for word, count in top_filtered]
counts = [count for word, count in top_filtered]
plt.figure(figsize=(9,6))
plt.barh(words[::-1], counts[::-1])
plt.xlabel("Frequency")
plt.ylabel("Words")
plt.title("Top 20 Most Frequent Words (After Stopword Removal)")
plt.show()
```

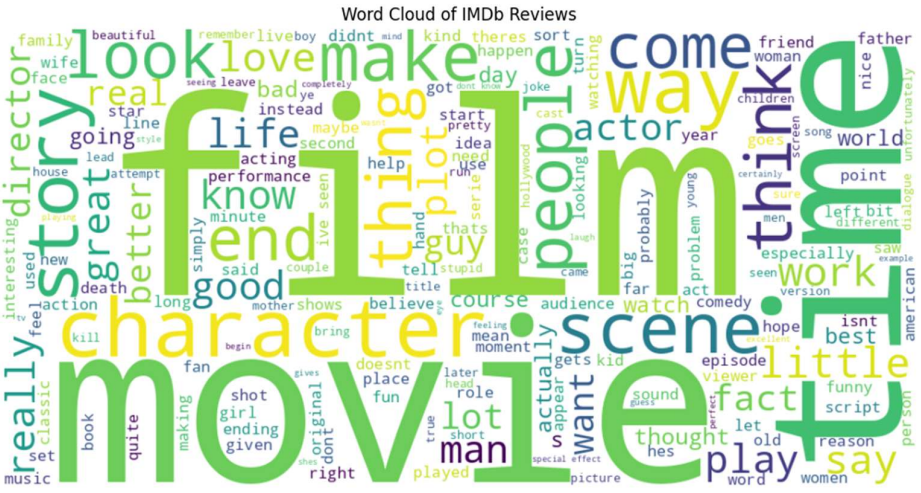
```
Performed By Girish Sakpal(UIN: 231A011)
```




```
print("Performed By Girish Sakpal(UIN: 231A011)")
clean_text_combined = " ".join(filtered_words)
wordcloud = WordCloud(
    width=1000,
    height=500,
    background_color="white",
).generate(clean_text_combined)

plt.figure(figsize=(12, 6))
plt.imshow(wordcloud, interpolation="bilinear")
plt.axis("off")
plt.title("Word Cloud of IMDb Reviews")
plt.show()
```

```
Performed By Girish Sakpal(UIN: 231A011)
```



```
print("Performed By Girish Sakpal(UIN: 231A011)")
#Top Bigrams
bigram_vectorizer = CountVectorizer(
    stop_words="english",
    ngram_range=(2, 2),
    max_features=20,
)
bigram_matrix = bigram_vectorizer.fit_transform(
    df["clean_text"])
bigram_counts = bigram_matrix.sum(axis=0).A1
bigram_names = bigram_vectorizer.get_feature_names_out()
bigram_df = pd.DataFrame(
    {"Bigram": bigram_names,
     "Frequency": bigram_counts}
)
bigram_df = bigram_df.sort_values(
    "Frequency", ascending=False
)

print("\nTop 20 Bigrams:")
print(bigram_df.head(20))
```

```
Performed By Girish Sakpal(UIN: 231A011)

Top 20 Bigrams:
             Bigram  Frequency
6          ive seen       3300
1         dont know       2193
15  special effects       2134
8        looks like       1661
11       movie just       1477
16       waste time       1425
5           im sure       1369
3        good movie       1363
17      watch movie       1296
2        dont think       1296
7         look like       1291
13         new york       1245
9        low budget       1224
19        years ago       1197
14      pretty good       1073
18         year old       1061
4       high school       1059
10   main character       1041
12       movie like       1038
0         bad movie        979
```

Conclusion:

The experiment successfully performed EDA on the IMDb text corpus by analyzing text statistics, word frequencies, stop words, word clouds, and bigrams. The visualizations helped identify important patterns and common terms, providing useful insights for further NLP tasks such as sentiment analysis and text classification.

CO's Covered:

CO1) Apply various text processing techniques

CO4) Realize semantics of English language for text processing